KRONOX 1914

gruppo-kronox

Simona Marinacci

Esame di

Metodi Avanzati di Programmazione

(track M-Z)

A.A. 2024/2025

Indice

Descrizione dell'avventura……pg.3
Architettura del Software e Scelte progettuali……pg.3
Progettazione……pg.5
Descrizione dei files json di configurazione del gioco……pg.15
Diagramma delle classi……pg.17
    Lato Client……pg.18
    Lato Server……pg.20
Specifica Algebrica: Room……pg.23
Dettagli implementativi……pg.28
    Programmazione Generica……pg.28
    File……pg.28
    DataBase(JDCB)……pg.29
    Lambda Expression……pg.29
    SWING……pg.30
    Thread e programmazione concorrente……pg.30
    Socket e/o REST……pg.30
Appendice……pg.31

Descrizione dell’avventura

Il presente documento descrive lo sviluppo e l'implementazione di "KRONOX 1914", un'applicazione software di tipo avventura testuale. Il progetto si pone il duplice obiettivo di creare un'esperienza narrativa coinvolgente per l'utente e, al contempo, di applicare concretamente i principi fondamentali della programmazione a oggetti, dell'ingegneria del software e della gestione dei dati.

L'applicazione, sviluppata in linguaggio **Java**, simula un'avventura in cui il giocatore viaggia nel tempo per risolvere un mistero, interagendo con l'ambiente attraverso comandi testuali. L'**architettura client-server** e la **persistenza dei dati su database** costituiscono i pilastri tecnici del progetto.

"KRONOX 1914" è un'avventura testuale in cui il giocatore assume il ruolo dell'assistente del Professor Vellini, un brillante informatico con una profonda passione per la Storia, la cui recente e improvvisa scomparsa è il punto di partenza del gioco. Il professore ha lasciato in eredità al protagonista il suo sapere e una macchina del tempo denominata Kronox, una rivelazione contenuta in una lettera ritrovata sulla scrivania dell'assistente nel suo ufficio. L'avventura inizia con il protagonista nel proprio ufficio, adiacente a quello del defunto Professor Vellini. Il viaggio ha inizio con la lettura di questa lettera e la successiva ricerca della macchina del tempo. Una volta scoperto il codice necessario per sbloccare la macchina Kronox, il protagonista intraprende un inaspettato viaggio nel tempo.
Questo viaggio conduce il protagonista, senza che ne comprenda immediatamente il motivo, al 27 giugno 1914. L'obiettivo è esplorare l'Ambasciata Vaticana a Sarajevo alle ore 09:00, per scoprire i segreti lasciati in eredità dal professore e forse far luce sui misteri legati allo scoppio della Prima Guerra Mondiale.

Architettura del Software e scelte progettuali

L'architettura del software è stata una delle decisioni più critiche per garantire la robustezza e la manutenibilità del progetto.

File JSON per la Configurazione Iniziale
Il **mondo di gioco** (stanze, oggetti, descrizioni, uscite) è definito in file esterni in formato JSON. Questa scelta disaccoppia i dati narrativi dalla logica di programmazione, permettendo di modificare o espandere il gioco senza dover ricompilare il codice Java.

Architettura Client-Server
La scelta di separare il **backend** (server) dal **frontend** (client) è stata motivata dalla necessità di scalabilità e separazione delle competenze.

- Il Server contiene l'Engine (motore di gioco), gestisce lo stato del mondo, la logica degli enigmi e la persistenza su database.
- Il Client è un'applicazione leggera responsabile unicamente di inviare l'input dell'utente al server e di mostrare l'output ricevuto. Questa architettura pone le basi per future espansioni, come un client grafico (GUI) o una modalità multiplayer.

Persistenza su Database H2
Per gestire i salvataggi e lo stato persistente degli oggetti, si è optato per un database relazionale embedded come H2. Questa scelta offre vantaggi significativi rispetto a un salvataggio su file di testo:

- **Robustezza**: Le transazioni garantiscono l'atomicità delle operazioni di salvataggio.
- **Struttura**: I dati sono organizzati in tabelle definite (SAVE_SLOTS, OBJECT_STATES), rendendo le query e la gestione più semplici.
- **Scalabilità**: Supporta in modo efficiente la gestione di numerosi oggetti e stati senza degrado delle performance.
-

Design Pattern Observer
Il cuore delle interazioni di gioco è gestito tramite il pattern Observer. Il motore di gioco (Engine) agisce come Observable, notificando una lista di Observer (es. PickUpObserver, UseObserver, MoveObserver) ogni volta che viene eseguito un comando. Ogni Observer è specializzato nel gestire un tipo di comando, rendendo il sistema estremamente modulare: per aggiungere un nuovo tipo di azione, è sufficiente creare una nuova classe Observer senza modificare il motore centrale.

Gestione dinamica dei comandi tramite Action Binder
Action Binder è una classe che gestisce le azioni del gioco. Essa usa una serie di if-else. Ogni if controlla se il comando inserito dall'utente corrisponde ad una azione specifica. Se c'è corrispondenza viene eseguita l'azione associata. Gli Observer invece sono classi che 'osservano' i cambiamenti nel gioco e reagiscono di conseguenza, fungendo da **dispatcher** per l'Action Binder.

Comunicazione di Rete: Scelta delle Java Sockets:
Nel progetto "KRONOX 1914", la comunicazione tra il Client e il Server è stata implementata utilizzando direttamente le Java Sockets. Questa scelta è stata preferita rispetto all'adozione di un'architettura basata sui metodi HTTP (GET, POST, ecc.) e sulle API RESTful, pur essendo quest'ultime una soluzione comune e potente per i sistemi distribuiti.
La decisione è stata guidata da due fattori specifici:

1. **Comunicazione Bidirezionale Persistente**: Un gioco interattivo, sebbene testuale, beneficia di un canale di comunicazione persistente dove sia il client che il server possono inviare dati in qualsiasi momento, senza la necessità di stabilire una nuova connessione per ogni singola operazione. Le Java Sockets offrono nativamente questo tipo di flusso di dati bidirezionale raw (InputStream e OutputStream), garantendo una gestione più fluida degli aggiornamenti di stato del gioco inviati dal server (es. avanzamento del tempo atmosferico tramite AtmosphereThread) e dei comandi del giocatore inviati dal client. Questo si contrappone al modello stateless di HTTP.
2. **Controllo a Basso Livello e Flessibilità del Protocollo**: L'uso delle Sockets ha fornito un controllo granulare sul protocollo di comunicazione. Non essendoci vincoli sulle convenzioni e sull'overhead degli header HTTP, si è potuto definire un protocollo personalizzato e snello, ottimizzato esclusivamente per lo scambio dei comandi parsati (ParserOutput) e dei messaggi testuali/strutturati (es. "INVENTORY_DATA") specifici del gioco. Questo ha permesso di ridurre l'overhead di rete e di concentrarci sulla logica di gioco senza le astrazioni aggiuntive di un framework RESTful.

In conclusione, la scelta delle Java Sockets ha rappresentato la soluzione più diretta ed efficiente per i requisiti di comunicazione dell'avventura testuale, fornendo il giusto bilanciamento tra flessibilità, controllo e complessità implementativa.

Progettazione

Il progetto "KRONOX1914" è organizzato in diversi pacchetti Java, ognuno con responsabilità specifiche, garantendo una chiara separazione delle preoccupazioni e una maggiore manutenibilità del codice.

adventure/

Questo pacchetto contiene le classi centrali che gestiscono la logica di gioco principale, l'engine e l'interazione client-server.

- **Client.java**:
  - **Contenuto**: Questa classe gestisce la logica lato client dell'applicazione, inclusa la connessione al server, l'interazione con l'interfaccia utente (GUI), l'invio dei comandi e l'elaborazione delle risposte. Implementa una macchina a stati (GameState) per gestire le diverse fasi del gioco e utilizza classi dedicate per l'interfaccia grafica e la riproduzione audio.
  - **Responsabilità**: Funge da punto di ingresso dell'applicazione client, stabilisce e mantiene la connessione con il server di gioco, e orchestra la comunicazione tra l'utente e il motore di gioco. Pre-processa l'input dell'utente e invia i comandi al server, aggiornando l'interfaccia utente in base ai messaggi ricevuti. Gestisce le transizioni tra i vari stati del gioco, controllando il flusso e il comportamento dell'applicazione client, e si occupa della gestione della disconnessione e della riproduzione della colonna sonora.
- **Engine.java**:
  - **Contenuto**: Questa classe gestisce il ciclo di gioco principale, interagisce con il GameDescription per la logica del gioco e con il Parser per l'interpretazione dei comandi. Si occupa anche del caricamento delle stopwords e della comunicazione con il client per gli aggiornamenti della stanza e le sequenze di gioco (intro/outro).
  - **Responsabilità**: È il cuore del motore del gioco, orchestra l'esecuzione del gioco dal caricamento iniziale alla fine, processando l'input del giocatore, aggiornando lo stato del gioco, e comunicando gli output rilevanti al client.
- **GameDescription.java**:
  - **Contenuto**: Una classe astratta che definisce la struttura di base di un'avventura testuale, inclusi stanze, comandi, inventario, la stanza corrente e l'epoca corrente. Dichiara metodi astratti per l'inizializzazione del gioco, l'elaborazione del prossimo comando, il messaggio di benvenuto, l'aggiornamento del mondo in base all'epoca, e la gestione della fine del gioco.
  - **Responsabilità**: Definisce il contratto e le proprietà fondamentali per qualsiasi gioco di avventura testuale basato su questa architettura, fornendo un'interfaccia standard per il motore di gioco e permettendo implementazioni specifiche del gioco stesso.

- **GameObservable.java**:
  - **Contenuto**: Un'interfaccia Java che definisce i metodi attach, detach e notifyObservers.
  - **Responsabilità**: Implementa il lato "Observable" del pattern Observer/Observable, fornendo un meccanismo per registrare, deregistrare e notificare gli oggetti GameObserver quando si verificano eventi rilevanti nel gioco.
- **GameObserver.java**:
  - **Contenuto**: Un'interfaccia Java che definisce un singolo metodo update che accetta un'istanza di GameDescription e ParserOutput.
  - **Responsabilità**: Implementa il lato "Observer" del pattern Observer/Observable, definendo il comportamento che le classi osservatrici devono seguire per reagire ai cambiamenti o agli eventi notificati da un oggetto GameObservable.
- **GameUtils.java**:
  - **Contenuto**: Contiene un metodo statico per recuperare un oggetto specifico dall'inventario del giocatore tramite il suo ID.
  - **Responsabilità**: Fornisce funzionalità di utilità per la manipolazione e l'accesso agli oggetti all'interno dell'inventario del gioco.
- **Server.java**:
  - **Contenuto**: La classe principale che avvia il server di gioco, gestisce le connessioni dei client, il caricamento/salvataggio dei progressi e l'esecuzione del motore di gioco (Engine) per ciascun client connesso. Interagisce con il database per la gestione degli slot di salvataggio.
  - **Responsabilità**: Inizia e mantiene il servizio di gioco sul lato server, gestisce l'accettazione di nuove connessioni client e coordina l'intera sessione di gioco per ciascun giocatore, inclusa la selezione e la gestione dei salvataggi (nuova partita, continua, sovrascrivi) e la chiusura delle risorse.

adventure.ui/

Questo pacchetto contiene le classi relative all'interfaccia utente grafica (GUI) del gioco, gestendo la visualizzazione e l'interazione con l'utente.

- **GameUI.java**:
  - **Contenuto**: Questa è la finestra principale del gioco, un JFrame che visualizza la stanza corrente (nome e immagine), l'output testuale del gioco e i controlli per l'input dell'utente (campo di testo e pulsanti). Gestisce anche l'aggiornamento della stanza, la visualizzazione di testi a schermo intero con effetto "macchina da scrivere" e le transizioni tra le schermate.
  - **Responsabilità**: Funge da interfaccia utente principale per il gioco, visualizzando lo stato corrente del gioco (descrizioni, immagini), accettando l'input del giocatore e presentando l'output del motore di gioco. Gestisce la logica di visualizzazione, inclusi effetti grafici come il testo a macchina da scrivere, e delega i comandi dell'utente alla classe Client.
- **ImageDialogUI.java**:
  - **Contenuto**: Una finestra di dialogo modale (JDialog) progettata per visualizzare un'immagine a schermo intero con un titolo. Carica

dinamicamente l'immagine dal percorso specificato e la scala per adattarla alla finestra.
    - **Responsabilità**: Visualizza immagini a schermo intero in una finestra di dialogo separata, permettendo al gioco di mostrare illustrazioni o elementi visivi specifici in momenti chiave.
- **InventoryDialogUI.java**:
    - **Contenuto**: Una finestra di dialogo modale (JDialog) che mostra il contenuto dell'inventario del giocatore. Popola dinamicamente il pannello con etichette per il nome e aree di testo per la descrizione di ogni oggetto nell'inventario.
    - **Responsabilità**: Fornisce una rappresentazione grafica dell'inventario del giocatore, elencando gli oggetti posseduti con i loro nomi e descrizioni, e permettendo al giocatore di visualizzarli.
- **ShowHelpCommandsDialogUI.java**:
    - **Contenuto**: Una finestra di dialogo modale (JDialog) che presenta un elenco formattato in HTML dei comandi disponibili nel gioco. Il contenuto dei comandi è predefinito nel codice HTML interno.
    - **Responsabilità**: Offre una guida in-game all'utente, mostrando un riepilogo dei comandi di movimento e azione che possono essere utilizzati, facilitando l'apprendimento delle interazioni di gioco.

constants/

Questo pacchetto è dedicato alla definizione di costanti globali utilizzate in tutto il progetto.

- **GameIDs.java**:
    - **Contenuto**: Questa classe di utilità contiene costanti statiche e finali di tipo intero e stringa che rappresentano identificatori univoci per tutti gli oggetti (LETTERA_PROFESSORE, CHIAVE_MAGAZZINO, ecc.) e le stanze (UFFICIO_ASSISTENTE_2025, INGRESSO_AMBASCIATA_1914, ecc.) presenti nel gioco. Definisce inoltre le costanti per le diverse epoche di gioco (PRESENT_EPOCH, PAST_EPOCH).
    - **Responsabilità**: Centralizza la definizione di tutti gli identificatori chiave del gioco, evitando l'uso di "Magic Numbers". Ciò rende il codice più leggibile, manutenibile e meno soggetto a errori, garantendo che le stesse entità siano sempre riferite con lo stesso ID in tutto il progetto.

database/

Questo pacchetto gestisce la persistenza dei dati di gioco, inclusi lo stato degli oggetti e i salvataggi, interagendo direttamente con il database.

- **DBManager.java**:
    - **Contenuto**: Questa classe statica gestisce la connessione al database H2, la sua creazione e inizializzazione (incluse le tabelle save_slots e object_state), e la chiusura della connessione. Definisce URL, utente e password del database e garantisce che lo schema del database sia configurato correttamente al primo avvio.
    - **Responsabilità**: Fornisce un punto centralizzato per la gestione della connessione al database, assicurando che le risorse siano correttamente allocate e deallocate. Si occupa della creazione dello schema del database e dell'inizializzazione degli slot di salvataggio al primo avvio dell'applicazione.

- **H2ObjectStateRepository.java**:
    - **Contenuto**: Implementazione specifica dell'interfaccia ObjectStateRepository per il database H2. Contiene metodi per caricare tutti gli stati degli oggetti di gioco da un dato slot, salvare lo stato attuale degli oggetti (inclusa la loro posizione e proprietà come "apribile", "spingibile", "bloccato") e inizializzare lo stato degli oggetti per una nuova partita.
    - **Responsabilità**: Gestisce la persistenza e il recupero dello stato dinamico degli oggetti di gioco nel database. Assicura che le modifiche agli oggetti durante il gioco (es. se una porta è aperta o chiusa) siano memorizzate e caricate correttamente per ciascun slot di salvataggio.
- **H2SaveSlotRepository.java**:
    - **Contenuto**: Implementazione delle interfacce Repository
    - **Responsabilità**: Gestisce la persistenza e il recupero dello stato complessivo di una partita (come la posizione del giocatore, l'epoca corrente e vari flag di gioco). Si occupa anche della gestione dei metadati degli slot di salvataggio, consentendo operazioni come la creazione di nuove partite, il caricamento di quelle esistenti e la gestione dello stato di completamento.
- **ObjectStateRepository.java**:
    - **Contenuto**: Un'interfaccia che definisce il contratto per la gestione dello stato degli oggetti nel database. Dichiara metodi per caricare, salvare e inizializzare gli stati degli oggetti.
    - **Responsabilità**: Stabilisce un'astrazione per l'accesso e la manipolazione dei dati relativi allo stato degli oggetti di gioco, promuovendo il disaccoppiamento tra la logica di business e la specifica implementazione del database.
- **Repository.java**:
    - **Contenuto**: Un'interfaccia generica che definisce le operazioni di base per un repository di dati: save e load. Utilizza un tipo generico
    - **Responsabilità**: Fornisce un'interfaccia comune per le classi che si occupano della persistenza dei dati, definendo un contratto generale per il salvataggio e il caricamento di entità.
- **SaveSlotRepository.java**:
    - **Contenuto**: Un'interfaccia che definisce le operazioni specifiche per la gestione dei metadati degli slot di salvataggio. Dichiara metodi per caricare tutti i metadati degli slot, aggiornare il nome di uno slot, marcare uno slot come completato e resettare uno slot.
    - **Responsabilità**: Definisce un contratto per la gestione delle informazioni relative ai punti di salvataggio del gioco, come il loro nome, la data dell'ultimo salvataggio e lo stato di completamento, separando queste operazioni dalla gestione dello stato di gioco effettivo.

exceptions/

Questo pacchetto contiene le classi di eccezione personalizzate del progetto, utilizzate per gestire errori specifici del gioco.

- **ActionException.java**:
    - **Contenuto**: Questa classe è un'estensione di GameException e viene utilizzata per rappresentare errori specifici legati al fallimento di un'azione del giocatore, come tentare di aprire una porta bloccata o usare un oggetto in

modo improprio. Fornisce costruttori per messaggi di errore semplici o per incapsulare un'altra eccezione.
    - **Responsabilità**: Segnala che un'azione tentata dal giocatore non può essere eseguita a causa di una ragione logica all'interno del gioco. Permette al motore di gioco di catturare e gestire specificamente questi tipi di errori, fornendo un feedback appropriato all'utente.
- **GameException.java**:
    - **Contenuto**: Questa classe base estende Exception e funge da superclasse per tutte le eccezioni personalizzate che possono verificarsi all'interno del gioco. Dispone di costruttori per accettare un messaggio di errore e per incapsulare un'altra eccezione (Throwable cause).
    - **Responsabilità**: Fornisce un tipo di eccezione generico per catturare e gestire tutti gli errori che si verificano nella logica del gioco. Essendo la base per altre eccezioni più specifiche, permette una gestione degli errori più granulare e una migliore tracciabilità dei problemi all'interno dell'applicazione.

implementazione/

Questo pacchetto contiene le implementazioni concrete delle logiche di gioco e degli "Observers" che reagiscono ai comandi e agli eventi di gioco.

- **ActionBinder.java**:
    - **Contenuto**: Questa classe statica centralizza la definizione e l'associazione di tutte le azioni personalizzate agli AdvObject del gioco. Utilizza espressioni lambda per definire il comportamento specifico quando un oggetto viene spinto (onPushAction), aperto (onOpenAction), osservato (onLookAction), si parla con esso (onTalkAction), o viene usato con un altro oggetto o del testo (onUseWithObjectAction, onUseWithTextAction). Contiene logiche complesse per i puzzle, le transizioni di stato del gioco (es. accendere luci, sbloccare passaggi, cambiare epoca) e l'aggiunta di indizi.
    - **Responsabilità**: Collega la logica di gioco specifica (puzzle, eventi narrativi, modifiche dello stato del mondo) agli oggetti interattivi. Facilita la creazione di comportamenti complessi per gli oggetti, mantenendo gli Observer generici e disaccoppiati dalla logica specifica del gioco.
- **InventoryObserver.java**:
    - **Contenuto**: Implementa l'interfaccia GameObserver e gestisce il comando INVENTORY. Prende la lista degli oggetti nell'inventario del giocatore, li formatta in una stringa specifica ("INVENTORY_DATA:nome|descrizione,...") e la restituisce affinché il client possa visualizzare un dialogo dell'inventario. La logica include la pulizia della descrizione degli oggetti per evitare problemi di parsing nel client.
    - **Responsabilità**: Fornisce al client i dati dell'inventario del giocatore in un formato standardizzato per la visualizzazione.
- **Kronox1914.java**:
    - **Contenuto**: Questa classe estende GameDescription e implementa GameObservable, costituendo l'implementazione specifica del gioco "Kronox 1914". Si occupa dell'inizializzazione del gioco (caricamento di stanze e oggetti da JSON, configurazione iniziale o caricamento da salvataggio), della gestione del prossimo movimento del giocatore, del salvataggio/caricamento

dello stato del gioco e degli oggetti tramite i repository del database, e della logica specifica dei puzzle e degli eventi narrativi (come il tempo che passa o lo stato della luce nel seminterrato). Registra anche tutti i GameObserver necessari per le azioni di gioco e avvia un thread per l'atmosfera.
    - **Responsabilità**: È l'implementazione concreta della logica del gioco "Kronox 1914". Gestisce l'intero ciclo di vita del gioco, dal setup iniziale alla gestione delle interazioni, dei puzzle, della persistenza dei dati e degli eventi narrativi, fungendo da hub per tutte le logiche specifiche del mondo di gioco.
- **LightObserver.java**:
    - **Contenuto**: Implementa l'interfaccia GameObserver e reagisce ai comandi TURN_ON o PUSH (specificamente su un interruttore). Contiene la logica per accendere o spegnere la luce in specifiche stanze (es. SCANTINATO_SEGRETO_2025) e aggiorna lo stato del gioco di conseguenza, inclusa la visibilità di oggetti e la descrizione della stanza.
    - **Responsabilità**: Gestisce le interazioni relative all'illuminazione nel gioco, permettendo al giocatore di alterare l'ambiente e sbloccare nuove aree o oggetti.
- **LookAtObserver.java**:
    - **Contenuto**: Implementa l'interfaccia GameObserver e gestisce il comando LOOK_AT. Se il comando riguarda un oggetto, verifica se l'oggetto ha un'azione personalizzata "onLookAction"; altrimenti, mostra la descrizione standard dell'oggetto, la sua immagine (se presente) e, se è un contenitore, il suo contenuto. Se il comando è "osserva stanza", restituisce la descrizione estesa della stanza corrente.
    - **Responsabilità**: Fornisce al giocatore descrizioni dettagliate di oggetti e ambienti, supportando sia descrizioni statiche che comportamenti dinamici definiti tramite azioni personalizzate sugli oggetti.
- **MoveObserver.java**:
    - **Contenuto**: Implementa l'interfaccia GameObserver e gestisce i comandi di movimento (NORD, SUD, EST, OVEST, UP, DOWN). Controlla le condizioni per il movimento, inclusi blocchi specifici per stanze o oggetti (es. statue, porte bloccate), e gestisce le transizioni tra le stanze. Include anche logiche per eventi narrativi specifici che si attivano con il movimento in determinate stanze.
    - **Responsabilità**: Gestisce la navigazione del giocatore tra le stanze, applicando le regole di movimento, i blocchi e gli effetti collaterali narrativi che dipendono dalla transizione tra gli ambienti.
- **OpenObserver.java**:
    - **Contenuto**: Implementa l'interfaccia GameObserver e gestisce il comando OPEN. Controlla se l'oggetto da aprire ha un'azione personalizzata "onOpenAction"; in assenza di essa, implementa una logica generica per aprire contenitori (AdvObjectContainer), svuotandoli degli oggetti e rendendoli accessibili. Gestisce anche i messaggi di errore per oggetti non apribili.
    - **Responsabilità**: Controlla l'interazione del giocatore con gli oggetti "apribili" nel gioco, permettendo l'accesso a nuovi oggetti o sbloccando percorsi.
- **PickUpObserver.java**:
    - **Contenuto**: Implementa l'interfaccia GameObserver e gestisce il comando PICK_UP. Verifica se un oggetto è "raccoglibile" in base alla sua proprietà

pickupable. Se lo è, lo rimuove dalla stanza corrente e lo aggiunge all'inventario del giocatore. Gestisce anche i messaggi di errore per oggetti non raccoglibili.
    - **Responsabilità**: Gestisce l'interazione del giocatore con gli oggetti raccoglibili, permettendo al giocatore di aggiungere elementi al proprio inventario e rimuoverli dall'ambiente.
- **PushObserver.java**:
    - **Contenuto**: Implementa l'interfaccia GameObserver e gestisce il comando PUSH. Controlla se l'oggetto da spingere ha un'azione personalizzata "onPushAction"; in assenza di essa, gestisce una logica di fallback per oggetti genericamente "spingibili", fornendo feedback all'utente.
    - **Responsabilità**: Controlla l'interazione del giocatore con gli oggetti "spingibili", attivando azioni specifiche o fornendo feedback sulle interazioni.
- **TalkObserver.java**:
    - **Contenuto**: Implementa l'interfaccia GameObserver e gestisce il comando TALK_TO. Verifica se un personaggio o un oggetto ha un'azione di dialogo personalizzata "onTalkAction" e, se presente, la esegue per gestire l'interazione.
    - **Responsabilità**: Facilita le interazioni dialogiche con personaggi o oggetti nel gioco, attivando risposte predefinite o catene di eventi narrativi.
- **UseObserver.java**:
    - **Contenuto**: Implementa l'interfaccia GameObserver e gestisce il comando USE. Supporta due scenari principali: l'uso di un oggetto dell'inventario su un oggetto nella stanza (invObj su roomObj) e l'uso di una stringa di testo su un oggetto (per input come password). Cerca e attiva azioni personalizzate definite per queste interazioni.
    - **Responsabilità**: Gestisce l'applicazione di un oggetto su un altro, o di un input testuale su un oggetto, permettendo la risoluzione di enigmi o l'attivazione di eventi complessi.

parser/

Questo pacchetto è dedicato all'analisi dei comandi dell'utente, trasformando l'input testuale in una struttura comprensibile dal motore di gioco.

- **Parser.java**:
    - **Contenuto**: Questa classe è responsabile dell'analisi sintattica dei comandi testuali inseriti dal giocatore. Utilizza un set di "stopwords" (parole comuni da ignorare) e elenchi di comandi e alias per identificare il verbo principale e gli eventuali oggetti o argomenti nel comando. Supporta comandi con uno o due argomenti e gestisce la logica per trovare gli oggetti nell'inventario o nella stanza corrente.
    - **Responsabilità**: Trasforma una stringa di input grezza (il comando dell'utente) in un oggetto ParserOutput strutturato, che il motore di gioco può facilmente interpretare ed eseguire. È fondamentale per la comprensione dell'intento del giocatore.
- **ParserOutput.java**:
    - **Contenuto**: Una classe semplice che incapsula il risultato dell'operazione di parsing. Contiene il tipo di comando identificato (CommandType), l'oggetto

principale su cui agire (AdvObject o AdvObjectContainer), e un eventuale secondo oggetto o una stringa di testo come argomento.
    - **Responsabilità**: Fornisce un contenitore strutturato per il risultato dell'analisi del comando, rendendo più agevole il passaggio delle informazioni dal parser al motore di gioco.

sound/

Questo pacchetto gestisce la riproduzione di musica ed effetti sonori all'interno del gioco.

- **MusicPlayer.java**:
    - **Contenuto**: Implementa l'interfaccia Runnable per permettere la riproduzione audio in un thread separato. Carica un file audio specificato dal percorso, lo riproduce in loop continuo e fornisce un meccanismo per fermare la riproduzione in modo pulito. Utilizza le API javax.sound.sampled per la gestione dell'audio.
    - **Responsabilità**: Gestisce la riproduzione della colonna sonora o di altri suoni di sottofondo in modo asincrono, evitando di bloccare il thread principale dell'applicazione. Assicura la corretta gestione del ciclo di vita della riproduzione audio.

type/

Questo pacchetto contiene le definizioni delle classi di "tipo" o "modello" che rappresentano le entità fondamentali del gioco, come oggetti, stanze, comandi e lo stato del gioco.

- **AdvObject.java**:
    - **Contenuto**: La classe base per tutti gli oggetti interagibili nell'avventura (es. chiavi, porte). Definisce proprietà come ID, nome, descrizione, alias, immagine e vari flag (openable, pickupable, pushable, useable, locked). Include anche un sistema flessibile di callback (Consumer e Function) per associare azioni personalizzate (onOpenAction, onPushAction, onLookAction, onTalkAction, onUseWithTextAction, onUseWithObjectActions) a specifiche interazioni con l'oggetto.
    - **Responsabilità**: Rappresenta e gestisce lo stato e il comportamento di un singolo oggetto di gioco. Permette agli oggetti di reagire a diverse interazioni del giocatore attraverso un sistema di azioni personalizzabili, disaccoppiando la logica specifica dell'oggetto dagli Observer generici.
- **AdvObjectContainer.java**:
    - **Contenuto**: Estende AdvObject aggiungendo la capacità di contenere una lista di altri AdvObject. Fornisce metodi per aggiungere e rimuovere oggetti dalla sua lista interna.
    - **Responsabilità**: Rappresenta oggetti che possono fungere da contenitori per altri oggetti (come bauli, zaini o inventari), permettendo la gestione di collezioni di oggetti all'interno del gioco.
- **Command.java**:
    - **Contenuto**: Rappresenta un comando di gioco riconosciuto dal parser, caratterizzato da un CommandType, un nome e un set di alias. Implementa i metodi hashCode ed equals per una corretta comparazione.

    - **Responsabilità**: Incapsula le informazioni su un comando che il giocatore può impartire, facilitando il riconoscimento e l'elaborazione dell'input da parte del parser e del motore di gioco.
- **CommandType.java**:
    - **Contenuto**: Un'enumerazione (enum) che definisce i tipi di comandi validi nel gioco, come END, INVENTORY, NORD, SOUTH, OPEN, PICK_UP, TALK_TO, USE, LOOK_AT, TURN_ON, DOWN, UP, e HELP.
    - **Responsabilità**: Fornisce un set predefinito e fortemente tipizzato dei comandi che il gioco è in grado di comprendere, migliorando la chiarezza e la manutenibilità del codice del parser e degli Observer.
- **GameEpoch.java**:
    - **Contenuto**: Un tipo di dato enumerativo (enum) che definisce le epoche temporali fondamentali del gioco, come PRESENTE e PASSATO.
    - **Responsabilità**: Fornisce un set predefinito e fortemente tipizzato dei valori possibili per l'epoca corrente del gioco e per l'epoca di appartenenza delle stanze, garantendo la sicurezza del tipo, la leggibilità e la manutenibilità del codice che gestisce le transizioni temporali e gli stati correlati alle epoche.
- **GameState.java**:
    - **Contenuto**: Una classe che modella lo stato complessivo di una partita in un dato momento. Include la posizione del giocatore (locationId), l'epoca corrente (currentEpoch), lo stato della luce nel seminterrato (basementLit) e vari flag booleani per gli indizi del criptex (foundP, foundA, foundX) e lo stato del tempo (pastMidday).
    - **Responsabilità**: Incapsula tutte le variabili di stato cruciali per il salvataggio e il caricamento di una partita, permettendo di ripristinare il gioco esattamente dal punto in cui era stato lasciato.
- **IndizioCriptex.java**:
    - **Contenuto**: Un'enumerazione (enum) semplice che definisce i tre indizi necessari per risolvere l'enigma del Criptex: P, A, X.
    - **Responsabilità**: Rappresenta in modo tipizzato i singoli indizi di un puzzle specifico del gioco, facilitando la gestione dello stato di avanzamento di tale enigma.
- **ObjectData.java**:
    - **Contenuto**: Una classe POJO (Plain Old Java Object) che rappresenta la struttura dei dati per un oggetto di gioco così come viene caricata da un file di configurazione (es. JSON). Contiene proprietà come ID, epoca, nome, descrizione, immagine, alias, e stringhe per pickupable, openable, pushable, useable, locked, e una lista di ID di oggetti che può contenere.
    - **Responsabilità**: Serve come modello per deserializzare i dati statici degli oggetti da fonti esterne (file JSON), fornendo un modello di configurazione iniziale per la creazione di istanze di AdvObject all'avvio del gioco.
- **ObjectState.java**:
    - **Contenuto**: Una classe che rappresenta lo stato dinamico di un singolo oggetto in un determinato slot di salvataggio. Contiene l'ID dell'oggetto, la sua posizione (location), e i suoi stati correnti relativi a openable, pushable, e locked.

    - **Responsabilità**: Memorizza le proprietà di un oggetto che possono cambiare durante il gioco (come se sia aperto/chiuso, spinto, o bloccato) per un dato salvataggio, permettendo la persistenza del suo stato attuale.
- **Room.java**:
    - **Contenuto**: Rappresenta una singola stanza all'interno dell'ambiente di gioco. Include proprietà come ID, string ID, epoca, nome, descrizione, una descrizione estesa (look), immagine, riferimenti alle stanze adiacenti (nord, sud, est, ovest, su, giù), una lista di AdvObject presenti nella stanza e una mappa di uscite bloccate. Fornisce anche un metodo getExit per navigare tra le stanze.
    - **Responsabilità**: Definisce la struttura e le proprietà di un ambiente di gioco, fungendo da nodo nel grafo delle stanze e gestendo la presenza di oggetti e le connessioni con altre stanze.
- **RoomData.java**:
    - **Contenuto**: Una classe POJO che definisce la struttura dei dati per una stanza di gioco così come viene caricata da un file di configurazione (es. JSON). Contiene proprietà come ID, epoca, nome, descrizione, look, immagine, una mappa delle uscite e una lista di ID di oggetti inizialmente presenti. Include anche una mappa di uscite bloccate.
    - **Responsabilità**: Serve come modello per deserializzare i dati statici delle stanze da fonti esterne, fornendo un modello di configurazione iniziale per la creazione di istanze di Room all'avvio del gioco.
- **SlotMetadata.java**:
    - **Contenuto**: Una classe che incapsula i metadati di uno slot di salvataggio, come l'ID dello slot, il nome assegnato dal giocatore, l'ultima data/ora di salvataggio e un flag che indica se la partita è stata completata.
    - **Responsabilità**: Fornisce un modo per visualizzare e gestire le informazioni relative ai singoli salvataggi nel menu di selezione, senza caricare l'intero stato del gioco.

utils/

Questo pacchetto contiene classi di utilità generiche che forniscono funzionalità di supporto non specifiche di un particolare modulo del gioco, ma utili in diverse parti dell'applicazione.

- **AtmosphereThread.java**:
    - **Contenuto**: Implementa l'interfaccia Runnable e opera in un thread separato. Ha il compito di stampare messaggi atmosferici o suggerimenti nella console a intervalli regolari (ogni 2 minuti), adattando il messaggio all'epoca corrente del gioco (presente o passato).
    - **Responsabilità**: Migliora l'esperienza di gioco fornendo un feedback periodico e tematico all'utente tramite la console, senza bloccare il flusso principale del gioco.
- **GameLoader.java**:
    - **Contenuto**: Questa classe è responsabile del caricamento dei dati di gioco (stanze e oggetti) da file JSON. Utilizza la libreria Gson per deserializzare i file JSON in mappe di oggetti RoomData e ObjectData, che fungono da modelli di configurazione iniziale per la creazione delle entità di gioco. Gestisce anche la chiusura sicura degli stream.

    - **Responsabilità**: Inizializza i dati statici del mondo di gioco leggendoli da file di configurazione esterni, disaccoppiando la definizione del mondo dalla logica del gioco.
- **Utils.java**:
    - **Contenuto**: Contiene metodi statici per operazioni di utilità generiche, come il caricamento di liste di stringhe da file o stream e la suddivisione di stringhe in token ignorando le stopwords.
    - **Responsabilità**: Fornisce funzionalità di supporto riutilizzabili per la lettura di dati da file/stream e per il pre-processing del testo, in particolare per la gestione delle "stopwords" nel parsing dei comandi.

Descrizione dei files json di configurazione del gioco

Il progetto "Kronox1914" utilizza due file JSON (rooms.json e objects.json) per definire in modo strutturato e separato dal codice la configurazione iniziale del mondo di gioco e degli oggetti. Questi file fungono da "blueprint" o "modelli di configurazione" per il GameLoader, permettendo di costruire il mondo di gioco in modo dinamico e flessibile.

1. **rooms.json**
   Definisce tutte le stanze (o ambienti) disponibili nel gioco, organizzate per epoch (epoca) e identificate da un stringId univoco. Ogni stanza è un oggetto JSON che descrive le sue proprietà statiche e le sue connessioni con altre stanze.

   Struttura di un oggetto stanza (RoomData):
   - **id** (Integer): Un identificatore numerico univoco per la stanza.
   - **epoch** (String) Indica l'epoca in cui la stanza esiste ("presente" o "passato"), fondamentale per la logica di viaggio nel tempo.
   - **name** (String): Il nome breve della stanza, visualizzato all'utente.
   - **description** (String): Una descrizione dettagliata della stanza quando il giocatore vi entra, spesso contenente suggerimenti o informazioni ambientali.
   - **look** (String): Una descrizione più estesa o focalizzata della stanza, attivata dal comando "osserva".
   - **image** (String): Il percorso relativo a un file immagine associato alla stanza, utilizzato per la visualizzazione grafica nell'interfaccia utente.
   - **exits** (Object/Map): Una mappa che definisce le uscite disponibili dalla stanza. Le chiavi sono le direzioni ("nord", "sud", "ovest", "est", "su", "giu") e i valori sono i **stringId** delle stanze a cui conducono.
   - **objects** (Array di Stringhe): Una lista di stringId degli oggetti che sono inizialmente presenti in quella stanza all'inizio del gioco.
   - **blockedExits** (Object/Map): Una mappa che specifica quali uscite sono bloccate e l'ID dell'oggetto che le blocca. Questo permette una gestione dinamica delle uscite.

   Ruolo nel Gioco
   rooms.json è essenziale per costruire il grafo del mondo di gioco. Il GameLoader legge questi dati per creare le istanze di Room che l'Engine e Kronox1914 utilizzeranno per la navigazione, la descrizione degli ambienti e la gestione degli oggetti presenti in essi.

2. **objects.json**
   Questo file JSON definisce tutti gli oggetti interagibili presenti nel gioco, identificati da un stringId univoco. Ogni oggetto è un oggetto JSON che specifica le sue proprietà e il suo comportamento di base.
   Struttura di un oggetto (ObjectData):
   - **id** (Integer): Un identificatore numerico univoco per l'oggetto.
   - name (String): Il nome breve dell'oggetto, utilizzato per l'interazione.
   - **description** (String): Una descrizione testuale dell'oggetto utilizzata per il comando osserva [oggetto].
   - **image** (String): Il percorso relativo a un file immagine associato all'oggetto, utilizzato per la visualizzazione grafica (jframe: ImageDialogUI).
   - **aliases** (Array di Stringhe): Un elenco di nomi alternativi o sinonimi con cui l'utente può fare riferimento all'oggetto (es. "busta", "messaggio" per "lettera").
   - **pickupable** (String): Indica se l'oggetto può essere raccolto. Può essere "true" o una stringa che rappresenta un messaggio di errore specifico se non raccoglibile (se previsto, altrimenti si usa il messaggio standard).
   - **openable** (String): Indica se l'oggetto può essere aperto. Può essere "true" o una stringa che rappresenta un messaggio di errore specifico (se previsto, altrimenti si usa il messaggio standard).
   - **pushable** (String): Indica se l'oggetto può essere spinto. Può essere "true" o una stringa che rappresenta un messaggio di errore specifico (se previsto, altrimenti si usa il messaggio standard).
   - **useable** (String): Indica se l'oggetto può essere "usato". Può essere "true" o una stringa di errore specifico (se previsto, altrimenti si usa il messaggio standard).
   - **locked** (String): Indica se un oggetto è bloccato/chiuso. Può essere "true" o una stringa che è il messaggio di blocco personalizzato (se previsto, altrimenti si usa il messaggio standard).
   - **contains** (Array di Stringhe): Una lista di stringId degli oggetti che questo oggetto contenitore contiene all'inizio del gioco.
   - **talkable** (String): Usato per i personaggi, indica se si può parlare con loro. Può essere "true" o una stringa che è il messaggio di blocco personalizzato (se previsto, altrimenti si usa il messaggio standard).

   Ruolo nel Gioco
   objects.json fornisce la definizione statica di tutti gli AdvObject del gioco. Il GameLoader utilizza questi dati per creare le istanze di AdvObject e AdvObjectContainer che popolano le stanze e l'inventario del giocatore, e che sono interagibili tramite i vari Observer e l'ActionBinder.

Diagramma delle classi

Seguono i diagrammi delle classi con le dovute semplificazioni.
Questi diagrammi si concentrano sulla definizione delle classi, delle loro responsabilità, dei loro attributi e metodi, e delle relazioni logiche tra di esse (ereditarietà, realizzazione, associazione, composizione). Descrivono "cosa" il sistema fa e "come" è strutturato a livello

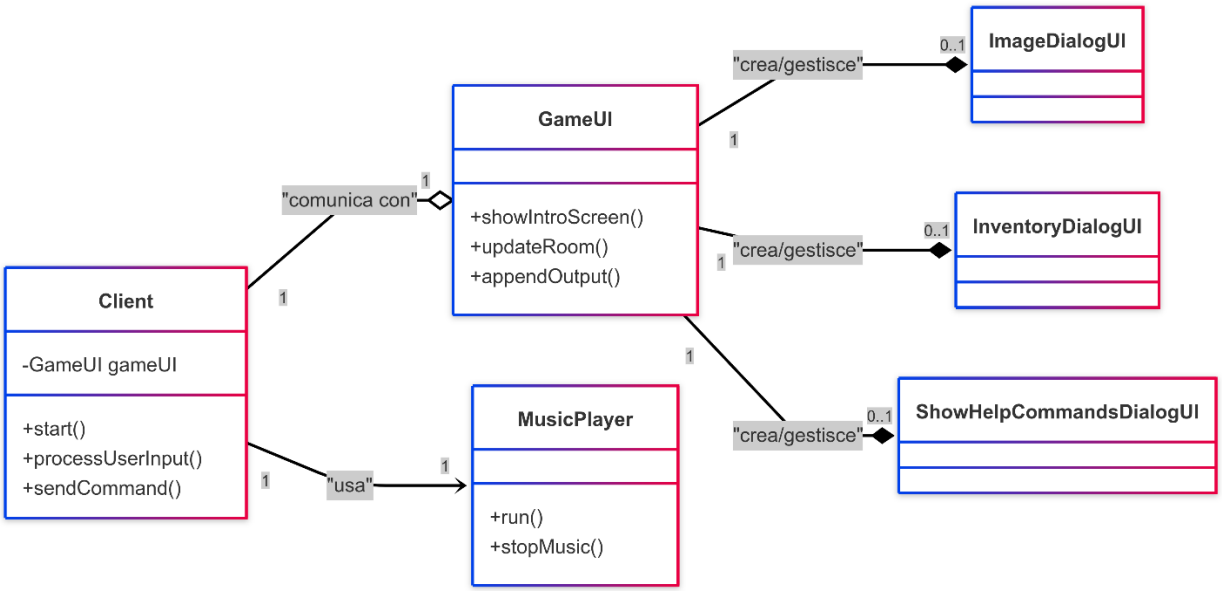
concettuale del codice, indipendentemente dall'ambiente di esecuzione fisico o dalla sua distribuzione.
Per offrire una comprensione più chiara dell'architettura del progetto "Kronox1914", il modello logico è stato suddiviso in due diagrammi delle classi significativi: uno focalizzato sul Lato Client e uno sulla Logica Software (Backend). Tale suddivisione permette di analizzare separatamente due macro-aree di complessità del sistema. Entrambi i diagrammi sono a livello logico.

Nel diagramma, le classi astratte sono contraddistinte dal loro nome in corsivo e dalla dicitura <<abstract>>. Similmente, le interfacce sono indicate con lo stereotipo <<interface>>. Per offrire una migliore visualizzazione, per rappresentarle sono stati utilizzati rispettivamente i colori rosa e viola.

Lo strumento utilizzato per la costruzione dei seguenti diagramma di classi è Mermaid diagram, dunque anche la notazione che viene descritta segue la notazione corrispondente alla documentazione del diagramma delle classi Mermaid.

Lato Client

Questo diagramma rappresenta la "vista" del sistema, ovvero le componenti con cui l'utente finale interagisce direttamente. È incentrato sull'interfaccia utente grafica e la logica di gestione della sessione lato client.



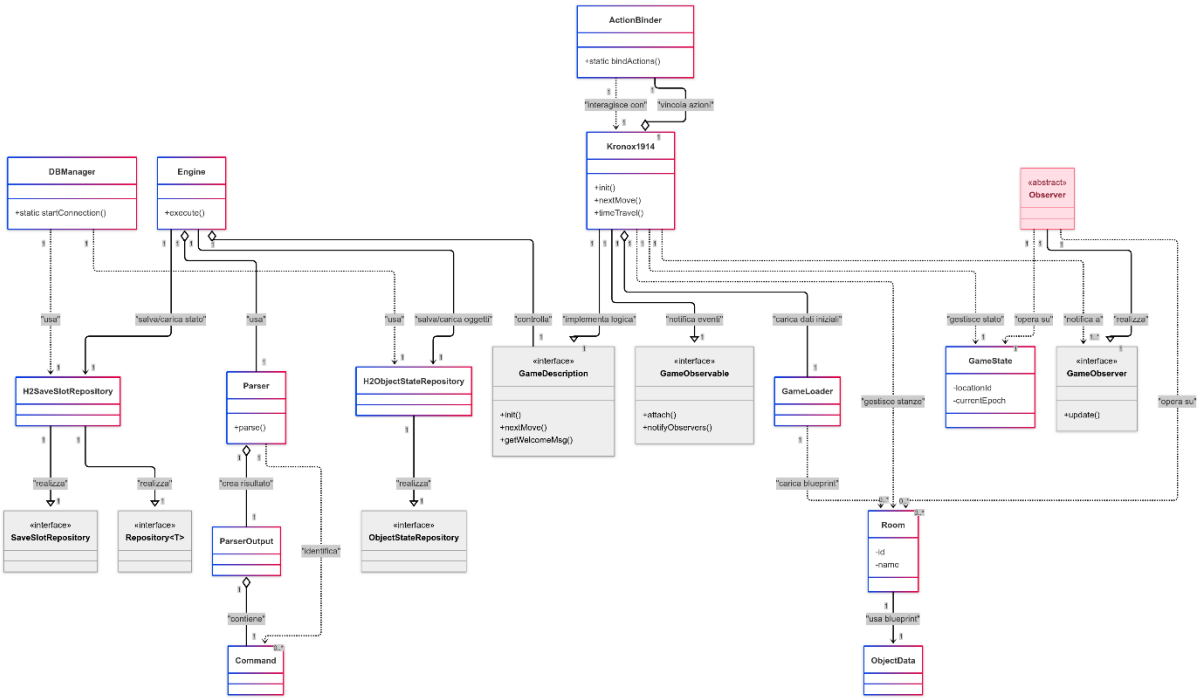
- **Client**: Questa classe rappresenta il punto di ingresso dell'applicazione lato utente. Le sue responsabilità includono la gestione dell'avvio del gioco (+start()), la connessione al server, l'elaborazione dell'input utente (+processUserInput()) e l'invio di comandi (+sendCommand()), oltre a orchestrare gli aggiornamenti dell'interfaccia. La relazione "comunica con" (1 a 1) tra Client e GameUI indica che Client utilizza e interagisce con l'interfaccia, mentre la dipendenza "usa" (1 a 1) da MusicPlayer mostra che Client è responsabile di avviare e gestire la musica di sottofondo.
- **GameUI**: Questa classe rappresenta la finestra principale del gioco. Gestisce la visualizzazione dell'output (testo, immagini) e l'acquisizione dell'input utente. Include metodi come +showIntroScreen(), +updateRoom(), e +appendOutput(). La GameUI ha una relazione di composizione (--*) con DialogUI (indicata dalla dicitura "crea/gestisce" e una molteplicità di 1 a 0..1 per ogni tipo di finestra di dialogo), il che significa che crea e gestisce finestre di dialogo secondarie e il loro ciclo di vita è fortemente legato a essa.
- **MusicPlayer**: Questa classe è utilizzata dal Client per la riproduzione audio, con metodi +run() e +stopMusic().
- **DialogUI** <<abstract>>: Questa è una classe astratta che funge da generalizzazione per tutte le finestre di dialogo secondarie, come quelle per l'inventario o le immagini. Da essa derivano:
    - ImageDialogUI: Una finestra di dialogo specifica per la visualizzazione di immagini.
    - InventoryDialogUI: Una finestra di dialogo specifica per mostrare il contenuto dell'inventario.
    - ShowHelpCommandsDialogUI: Una finestra di dialogo specifica per visualizzare i comandi di aiuto.

Ecco come le tecniche OO sono state menzionate o sono implicite nella descrizione:

- **Incapsulamento:** Descritto nella responsabilità di ogni classe, ad esempio, Client "Gestisce l'avvio del gioco, la connessione al server, l'elaborazione dell'input utente...". Questo implica che i dati e i metodi correlati sono racchiusi all'interno della classe.
- **Astrazione:** Evidente con la classe DialogUI <<abstract>>, che "È una classe astratta che funge da generalizzazione per tutte le finestre di dialogo secondarie". Questo mostra come si definisce un'interfaccia comune per un gruppo di classi correlate senza specificare tutti i dettagli di implementazione.
- **Ereditarietà (o Generalizzazione/Specializzazione):** Mostrata chiaramente dal fatto che ImageDialogUI, InventoryDialogUI, e ShowHelpCommandsDialogUI derivano da DialogUI <<abstract>>.
- **Polimorfismo:** Anche se non esplicitamente nominato, il polimorfismo è implicito nell'uso della classe astratta DialogUI. Diverse implementazioni (ImageDialogUI, InventoryDialogUI, ecc.) possono essere trattate in modo uniforme come oggetti DialogUI.
- **Composizione/Aggregazione:** Indicata dalla dicitura "crea/gestisce" tra GameUI e DialogUI. Questo significa che GameUI "crea e gestisce queste finestre di dialogo, e il loro ciclo di vita è fortemente legato a essa", suggerendo una composizione forte.
- **Associazione (o Dipendenza/Collaborazione):** La relazione "comunica con" tra Client e GameUI e "usa" tra Client e MusicPlayer sono esempi di associazione/dipendenza, indicando come le classi collaborano tra loro.

Lato Server

Questo diagramma illustra l'architettura del backend del gioco, focalizzandosi su come il motore elabora le informazioni, gestisce lo stato e interagisce con la persistenza dei dati.



Se l'immagine risulta poco visibile in questo file, visualizzarla all'interno della sottocartella images/ che si trova nello stesso path di questo stesso file

Segue la spiegazione dettagliata del diagramma delle classi di cui sopra con l'aggiunta di particolari talvolta non realmente visibili nel digramma stesso e omessi per il bisogno di leggibilità.

- **Engine**: Agisce come il direttore d'orchestra principale del gioco. Controlla una singola istanza di GameDescription (l'interfaccia del gioco). È responsabile di salvare e caricare lo stato del gioco e degli oggetti tramite H2SaveSlotRepository e H2ObjectStateRepository.
- **Kronox1914**: È l'implementazione concreta della logica di gioco. Kronox1914 "1" --|> "1" GameDescription indica che implementa il contratto definito da GameDescription. Analogamente, Kronox1914 "1" --|> "1" GameObservable mostra che implementa la capacità di notificare gli osservatori di eventi di gioco. Kronox1914 gestisce una singola istanza di ActionBinder (che vincola azioni agli oggetti), una singola istanza di GameLoader (per caricare i dati iniziali) e una singola AtmosphereThread (per gli eventi in background). Notifica a uno o più GameObserver che sono registrati. Inoltre, gestisce lo stato (GameState), le Room e gli AdvObject.
- **GameDescription**: Definisce il contratto base per qualsiasi gioco di avventura, fornendo l'interfaccia che l'Engine si aspetta per poter funzionare.
- **GameObservable** e GameObserver: Implementano il pattern Observer. **GameObservable** è il "soggetto" che gli Observer "osservano".

- **AdvObject**: Rappresenta un oggetto generico nel gioco, con attributi come ID, nome e stato di blocco.
- **AdvObjectContainer**: È una specializzazione di AdvObject che può contenere altri oggetti.
- **Room**: Definisce una stanza nell'ambiente di gioco. La relazione Room "1" o-- "*" AdvObject indica che una stanza contiene zero o molti AdvObject (composizione).
- **GameState**: Incapsula lo stato dinamico dell'intera partita, inclusa la posizione del giocatore e l'epoca corrente.
- **Command**: Rappresenta un comando di gioco. Command "1" o-- "0..*" AdvObject indica che un comando può riferirsi a zero o più AdvObject come argomenti.
- **ObjectData**: Rappresenta i dati di configurazione statici, anche chiamati blueprint di un oggetto. Room "1" --> "1" ObjectData indica che una stanza usa i dati di configurazione per i suoi oggetti iniziali.
- **ObjectState**: Rappresenta lo stato dinamico di un singolo oggetto. AdvObject "1" --> "0..1" ObjectState indica che un AdvObject ha al più un ObjectState.
- **DBManager**: Gestisce la connessione al database. Le dipendenze a H2SaveSlotRepository e H2ObjectStateRepository mostrano che ne fa uso. **DBManager** "1" ..> "1" GameException indica che può sollevare un'eccezione di gioco.
- **Repository, SaveSlotRepository** e **ObjectStateRepository**: Interfacce più specifiche per la gestione dei metadati degli slot e dello stato degli oggetti.
- **H2SaveSlotRepository** e **H2ObjectStateRepository**: Sono le implementazioni concrete per il database H2. Ciascuna realizza le rispettive interfacce, come H2SaveSlotRepository "1" --|> "1" Repository
- **ActionBinder**: Collega le azioni specifiche del gioco agli oggetti.
- **Observer**: Una classe astratta che rappresenta la generalizzazione di tutti gli specifici osservatori di gioco. Observer "1" --|> "1" GameObserver indica che implementa l'interfaccia GameObserver. Le sue dipendenze mostrano che opera su GameState, AdvObject, e Room.
- **Parser** e **ParserOutput**: Gestiscono l'analisi dell'input dell'utente. Parser "1" o-- "1" ParserOutput indica che il Parser crea un ParserOutput. ParserOutput "1" o-- "1" Command mostra che l'output del parser contiene un singolo Command.
- **GameLoader**: Carica i dati di configurazione iniziali di stanze e oggetti dal JSON. GameLoader "1" ..> "0.." *Room e GameLoader "1" ..> "0..*" AdvObject indicano che carica molti modelli di configurazione per stanze e oggetti.
- **GameException**: La classe base per tutte le eccezioni di gioco.

Le tecniche di programmazione orientata agli oggetti (OO) utilizzate nel progetto "KRONOX 1914", come dedotto dalla relazione e dai diagrammi delle classi, includono:

- **Programmazione a Oggetti (OOP)**: Il progetto è stato sviluppato in linguaggio Java e si proponeva di applicare estensivamente i principi della OOP. Questo è evidente nella struttura del codice, dove entità del mondo di gioco come stanze (Room), oggetti (AdvObject), comandi (Command) e lo stato del gioco (GameState) sono rappresentate come classi.
- **Incapsulamento**: Ogni classe incapsula i propri dati (attributi) e i metodi che operano su tali dati. Ad esempio, la classe Client gestisce la logica lato client, la connessione al server e l'elaborazione dell'input utente. Analogamente, Engine orchestra il ciclo di gioco principale. Questo principio è fondamentale per garantire

che lo stato interno degli oggetti sia controllato e modificabile solo attraverso interfacce ben definite.

- **Astrazione**: Utilizzata per definire interfacce comuni e classi base.
    - **Classi astratte**: GameDescription è una classe astratta che definisce il contratto base per qualsiasi gioco di avventura, fornendo un'interfaccia standard per il motore di gioco. Anche DialogUI è una classe astratta che generalizza le finestre di dialogo secondarie.
    - **Interfacce**: GameObservable e GameObserver definiscono il contratto per il pattern Observer, disaccoppiando il soggetto dagli osservatori. Le interfacce Repository<T>, SaveSlotRepository e ObjectStateRepository astraggono le operazioni di persistenza dei dati.
- **Ereditarietà (o Generalizzazione/Specializzazione)**: Permette alle classi di specializzarsi e riutilizzare il comportamento di classi più generiche.
    - Kronox1914 estende GameDescription e implementa GameObservable, ereditando e fornendo l'implementazione specifica della logica di gioco.
    - AdvObjectContainer estende AdvObject, aggiungendo la capacità di contenere altri oggetti.
    - Le classi ImageDialogUI, InventoryDialogUI, e ShowHelpCommandsDialogUI sono specializzazioni della classe astratta DialogUI.
- **Polimorfismo**: Implicito nell'uso delle interfacce e delle classi astratte. Diverse implementazioni di GameObserver (es. MoveObserver, PickUpObserver, UseObserver) possono essere trattate in modo uniforme dal GameObservable (Kronox1914) quando notifica gli eventi. Allo stesso modo, le diverse finestre di dialogo (ImageDialogUI, InventoryDialogUI, ecc.) possono essere gestite polimorficamente attraverso il riferimento alla loro superclasse astratta DialogUI.
- **Associazione/Collaborazione**: Le classi interagiscono tra loro per raggiungere gli obiettivi del sistema. Ad esempio, il Client comunica con la GameUI e utilizza il MusicPlayer. Il DBManager dipende dai repository per le operazioni sul database. Il Parser crea un ParserOutput che contiene un Command.
- **Composizione**: Indica una relazione forte "parte-di" dove una classe è composta da istanze di altre classi. Ad esempio, una Room contiene zero o molti AdvObject. La GameUI "crea/gestisce" le istanze di DialogUI (come ImageDialogUI o InventoryDialogUI), indicando che il loro ciclo di vita è legato a quello della GameUI.
- **Design Pattern Observer**: È il cuore delle interazioni di gioco. Il motore di gioco (Engine che notifica GameDescription/Kronox1914) agisce come *Observable* (soggetto), notificando una lista di *Observer* (es. PickUpObserver, UseObserver, MoveObserver) ogni volta che viene eseguito un comando. Ogni Observer è specializzato nel gestire un tipo di comando, rendendo il sistema estremamente modulare e facilmente estensibile.

Natura di semplificazione

È fondamentale comprendere che questi due diagrammi delle classi sono una semplificazione del sistema. Non includono ogni singolo attributo o metodo di ogni classe, né tutte le possibili dipendenze che potrebbero esistere nel codice sorgente completo. L'obiettivo è presentare il diagramma delle classi più significativo, che cattura l'essenza dell'architettura e le relazioni fondamentali, offrendo una comprensione chiara del design del software senza essere eccessivamente dettagliato.

Specifica algebrica: Room (Stanza del gioco)

La classe Room rappresenta una struttura dati astratta (ADT) per una singola stanza nel mondo di gioco di Kronox1914. Essa definisce la sua identità, descrizione, epoca di appartenenza, collegamenti con altre stanze e la gestione degli oggetti al suo interno.

1. Specifica Sintattica:

Sorts:

- **Room**,
- **Id** (Integer),
- **StringId** (String),
- **Name** (String),
- **Description** (String),
- **LookDescription** (String),
- **ImagePath** (String),
- **GameEpoch** (Enum{PRESENTE, PASSATO}),
- **Direction** (String),
- **ListAdvObject** (un ArrayList di AdvObject),
- **MapBlockedExits** (alias per il tipo di Map con <String, String> per le uscite bloccate, con chiave la direzione dell'uscita bloccata e valore il nome della variabile con cui è stato dichiarato l'oggetto che la blocca.),
- **Boolean**.

Nota bene: StringId, Name, Description, LookDescription, ImagePath e Direction, sono tutti alias del tipo String indicati ciascuno con il proprio alias per differenziare i ruoli di ciascun parametro.

Operations:

- **Costruttori**:
  - CreateRoom (Id, StringId, Name, Description) → Room
    - Crea una nuova stanza con l'ID numerico, l'ID testuale, il nome e la descrizione specificati.
  - setNorth (Room, Room)→ Room
  - setSouth (Room, Room) → Room
  - setEast (Room, Room) → Room
  - setWest (Room, Room) → Room
  - setUp (Room, Room) → Room
  - setDown (Room, Room) → Room
    - Questi operatori "setter" (come `setNorth`, `setSouth`, ecc.) sono qui interpretati come **costruttori**. Essi producono una **nuova istanza di**

Room con l'uscita specifica impostata, a partire da un'istanza room esistente. Il primo argomento è l'istanza della stanza di partenza, mentre il secondo argomento è la stanza di destinazione che verrà collegata nella direzione indicata dal nome dell'operazione (`North`, `South`, ecc.). Questa astrazione permette di rappresentare la modifica dello stato della stanza in un contesto algebrico, pur sapendo che nell'implementazione Java il metodo modifica l'istanza esistente.

- Per evitare ripetizioni e ridondanza nella specifica semantica, possiamo generalizzare questi setter di direzione. L'operatore `setDirection` rappresenta qualsiasi metodo setter di direzione (`setNorth`, `setSouth`, `setEast`, `setWest`, `setUp`, `setDown`). Il comportamento di base è lo stesso: dato un room e un `neighbor`, l'operazione collega room a `neighbor` nella direzione specificata dal nome del setter. Il valore in uscita (la nuova istanza di Room con la modifica applicata) seguirà sempre la stessa logica di trasformazione.

- setEpoch (Room, GameEpoch) → Room
- setLook (Room, LookDescription) → Room
- setImage (Room, ImagePath) → Room
- setBlockedExits (Room, MapBlockedExits) → Room
    - Questi altri operatori "setter" sono interpretati come costruttori che producono una *nuova istanza* di Room con la specifica proprietà impostata.

- **Osservatori:**
    - getID (Room) → Id
        - Restituisce l'ID numerico della stanza.
    - getStringID (Room)→ StringId
        - Restituisce l'ID testuale della stanza.
    - getName (Room) → Name
        - Restituisce il nome della stanza.
    - getDescription (Room)→ Description
        - Restituisce la descrizione breve della stanza.
    - getLook (Room) → LookDescription
        - Restituisce la descrizione estesa della stanza (per il comando "osserva stanza").
    - getImage (Room) → ImagePath
        - Restituisce il percorso dell'immagine della stanza.
    - getEpoch (Room) → GameEpoch
        - Restituisce l'epoca temporale a cui appartiene la stanza.
    - getObjects (Room) → ListAdvObject

        - Restituisce la lista degli oggetti presenti in questa stanza.

          Nota: Le operazioni di aggiunta/rimozione di oggetti dalla lista (add, remove, contains) sono operazioni dell'ADT ListAdvObject, non di Room.
    - getExit (Room, Direction) → Room (or null)
        - Restituisce la stanza collegata all'uscita specificata.
    - getBlockedExits (Room) → MapBlockedExits
        - Restituisce la mappa delle uscite bloccate.
    - hashCode (Room) → Integer
        - Restituisce il codice hash della stanza (basato sull'ID).
    - equals (Room, Room) → Boolean
        - Verifica l'uguaglianza tra due stanze (basata sull'ID).

2. Specifica Semantica (Assiomi):

Useremo **r**, **r'**, **r1** per Room, **id** per ID, **sid** per StringID, **n** per Name, **d** per Description, **l** per LookDescription, **img** per ImagePath, **e** per Epoch, **dir** per Direction, **obj**, **obj1** per AdvObject, **map** per MapBlockedExits, **neigh** per Room.

Osservazioni	**Costruttori di r'**					
	CreateRoom (id, sid, n, d)	**setDirection (r, neigh)**	**setEpoch (r, e)**	**setLook (r, l)**	**setImage (r, img)**	**setBlockedExits (r, map)**
getId (r')	id	getId (r)	getId (r)	getId (r)	getId (r)	getId(r)
getStringId (r')	sid	getStringId(r)	getStringId (r)	getStringId (r)	getStringId (r)	getStringId (r)
getName (r')	n	getName (r)	getName (r)	getName (r)	getName (r)	getName(r)
get Description (r')	d	getDescription (r)	getDescription (r)	getDescription (r)	getDescription (r)	getDescription (r)
getLook (r')	null	getLook (r)	getLook (r)	l	getLook (r)	getLook (r)
getImage (r')	null	getImage (r)	getImage (r)	getImage (r)	img	getImage (r)
getEpoch (r')	null	getEpoch(r)	e	getEpoch (r)	getEpoch (r)	getEpoch (r)
getObjects (r')	ListAdvObject. emptyList()	getObjects(r)	getObjects (r)	getObjects (r)	getObjects (r)	getObjects (r)
getExit (r', dir)	null	If dir == [direzione_del_setter] then neight else getExit (r, dir)	getExit (r, dir)	getExit (r, dir)	getExit (r, dir)	getExit (r, dir)
getBlockedExits (r')	MapBlockedExits. emptyMap ()	getBlockedExits (r)	getBlockedExits (r)	getBlockedExits (r)	getBlockedExits (r)	map
hashCode (r')	id	hashCode (r)	hashCode (r)	hashCode (r)	hashCode (r)	hashCode(r)
equals (r', r1)	If id == id1 then true else false	equals(r, r1)	equals (r, r1)	equals (r, r1)	equals (r, r1)	equals (r, r1)

id1 = getId (r1)
r1 = CreateRoom (id1, sid1, n1, d1)

Nota Dettagliata per l'Assioma getExit (r', dir) sotto il costruttore setDirection (r, neigh):

Nel contesto della specifica algebrica, `setDirection(r, neigh)` è un operatore costruttore utilizzato per rappresentare in modo conciso i sette operatori specifici di impostazione delle uscite presenti nella classe Room.java: `setNorth(r, neigh)`,

`setSouth(r, neigh)`, `setEast(r, neigh)`, `setWest(r, neigh)`, `setUp(r, neigh)`, e **`setDown(r, neigh)`**.
L'assioma per **`getExit(r', dir)`** sotto questa colonna definisce il comportamento dell'osservatore getExit quando applicato a una stanza r' che è stata creata utilizzando uno di questi setter direzionali.
L'espressione nella cella: `if dir == [direzione_del_setter] then neigh else getExit(r, dir)`, va interpretata come una generalizzazione delle seguenti sette regole specifiche:

- Quando il costruttore è `setNorth(r, northNeigh)`:
    - `getExit (setNorth(r, neigh), dir):`

      `if dir == "north" then neigh else getExit(r, dir)`
      (Se cerco l'uscita "nord" dalla stanza modificata con setNorth, ottengo neigh; altrimenti ottengo l'uscita dalla stanza r originale).
- Quando il costruttore è `setSouth(r, southNeigh)`:
    - `getExit (setSouth(r, neigh), dir):`

      `if dir == "south" then neigh else getExit(r, dir)`
- Quando il costruttore è `setEast(r, neigh)`:
    - `getExit (setEast(r, neigh), dir):`

      `if dir == "east" then neigh else getExit(r, dir)`
- Quando il costruttore è `setWest(r, neigh)`:
    - `getExit (setWest(r, neigh), dir):`

      `if dir == "west" then neigh else getExit(r, dir)`
- Quando il costruttore è setUp(r, neigh):
    - getExit (setUp(r, neigh), dir):

      if dir == "su" then neigh else getExit(r, dir)
- Quando il costruttore è `setDown(r, neigh)`:
    - `getExit (setDown(r, neigh), dir):`

      `if dir == "giu" then neigh else getExit(r, dir)`

3. Specifica di Restrizione:

- Unicità ID: L'operazione createRoom implica che l'ID numerico (id) e l'ID testuale (StringId) siano univoci per ogni stanza.
- Validità delle Direzioni: L'operazione getExit è definita solo per i valori di Direction specificati ("north", "south", "east", "west", "su", "giu"). Qualsiasi altra stringa di direzione risulterà in null.
- Coerenza del Grafo: Questa specifica algebrica dell'ADT Room non impone intrinsecamente che se la Stanza A ha un'uscita a Nord verso la Stanza B, allora la Stanza B abbia un'uscita a Sud verso la Stanza A. Questo è un vincolo di coerenza globale del grafo di gioco che deve essere garantito da un livello superiore.

Dettagli implementativi

Per ciascun argomento del corso spiegare se e come è stato utilizzato all'interno del progetto.

Programmazione Generica (Generics)

La programmazione generica è stata utilizzata nel progetto "KRONOX 1914" per migliorare la riusabilità del codice e la sicurezza dei tipi, in particolare nella gestione dei repository per la persistenza dei dati.

- **Interfaccia Repository<T>**: Il concetto di programmazione generica è evidente nell'interfaccia Repository<T>. Questa interfaccia definisce le operazioni di base di salvataggio (save) e caricamento (load) in modo indipendente dal tipo specifico di dato che deve essere persistito. L'uso del tipo generico <T> indica che questa interfaccia può operare su qualsiasi tipo di oggetto, rendendola altamente riutilizzabile per diversi tipi di entità di gioco che necessitano di essere salvate e caricate.
- **Implementazioni Specifiche dei Repository**: Le implementazioni concrete di questa interfaccia, come H2SaveSlotRepository e H2ObjectStateRepository, realizzano Repository<GameState> o altre interfacce specifiche che a loro volta possono estendere Repository<T>. Questo permette di avere una logica di persistenza unificata, ma con la flessibilità di gestire oggetti di tipo GameState o stati di oggetti in modo specifico, senza dover duplicare il codice per le operazioni di base di salvataggio e caricamento per ciascun tipo.
- **Vantaggi**: L'applicazione della programmazione generica in questo contesto ha garantito che le operazioni di persistenza fossero fortemente tipizzate al momento della compilazione, riducendo il rischio di errori a runtime legati a casting impropri o a tipi non corrispondenti.

File

Nel progetto "KRONOX 1914", i file sono stati impiegati principalmente per la **configurazione iniziale del mondo di gioco**.

- **Definizione del mondo di gioco**: Il mondo di gioco, incluse stanze, oggetti, descrizioni e uscite, è definito in file esterni in formato JSON, specificatamente rooms.json e objects.json. Questa scelta progettuale disaccoppia i dati narrativi dalla logica di programmazione, consentendo di modificare o espandere il gioco senza la necessità di ricompilare il codice Java.
- **Caricamento dei dati**: La classe GameLoader (utils/GameLoader.java) è responsabile del caricamento di questi dati da file JSON. Essa utilizza librerie esterne, come Gson, per deserializzare i file JSON in mappe di oggetti Java (RoomData e ObjectData), che fungono da modelli di configurazione iniziale.
- **Gestione I/O**: Per la lettura di questi file di testo, il progetto fa uso degli **Character Stream** e delle classi orientate alla riga, come BufferedReader e PrintWriter, per ottimizzare le operazioni di I/O grazie all'uso di un buffer. La chiusura degli stream è gestita in modo opportuno per evitare perdite di dati e liberare risorse. Sebbene il progetto utilizzi questi stream per i file di configurazione, la persistenza dello stato di gioco è delegata a un database per maggiore robustezza.

Database (JDBC)

La gestione dei dati persistenti e dei salvataggi in "KRONOX 1914" è un pilastro tecnico fondamentale, realizzata tramite un database relazionale.

- **Persistenza dei dati**: Il progetto implementa un sistema di persistenza dei dati utilizzando il database embedded **H2** per gestire i salvataggi del gioco e lo stato degli oggetti. Questa scelta è stata preferita rispetto al salvataggio su file di testo per la sua robustezza (transazioni garantiscono l'atomicità), la struttura organizzata dei dati in tabelle (SAVE_SLOTS, OBJECT_STATES) e la scalabilità.
- **JDBC (Java Database Connectivity)**: L'interazione con il database avviene tramite JDBC, lo standard Java per l'accesso ai database.
    - **Connessione**: La classe DBManager (database/DBManager.java) è statica e gestisce la connessione al database H2, inclusa la sua creazione e inizializzazione delle tabelle al primo avvio. Fornisce un punto centralizzato per la gestione della connessione. La connessione viene stabilita tramite il metodo getConnection della classe DriverManager.
    - **Repository**: Le operazioni di salvataggio e caricamento dello stato del gioco (GameState) e degli oggetti (ObjectState) sono gestite tramite classi repository specifiche: H2SaveSlotRepository e H2ObjectStateRepository. Queste classi implementano interfacce generiche (Repository<T>, SaveSlotRepository, ObjectStateRepository) per le operazioni di persistenza, promuovendo il disaccoppiamento dalla specifica implementazione del database.
    - **Query SQL**: Le query SQL per modificare (executeUpdate) o interrogare (executeQuery) il database sono eseguite tramite oggetti Statement o PreparedStatement, ottenuti dall'oggetto Connection. Eventuali errori durante le interrogazioni sul database sollevano eccezioni di tipo SQLException.

Lamba Expression (compresi stream e pipeline)

Le Lambda Expression sono state adottate nel progetto per migliorare la concisione e la flessibilità del codice, specialmente nell'assegnazione di comportamenti dinamici.

- **ActionBinder**: La classe ActionBinder (implementazione/ActionBinder.java) è un esempio chiave dell'uso delle espressioni lambda. Centralizza la definizione e l'associazione di azioni personalizzate agli AdvObject del gioco, utilizzando espressioni lambda per definire il comportamento specifico di un oggetto quando viene spinto (onPushAction), aperto (onOpenAction), osservato (onLookAction), o usato (onUseWithObjectAction, onUseWithTextAction). Le lambda offrono un modo conciso per implementare le interfacce funzionali, come quelle che definiscono le azioni degli oggetti.

SWING

Java Swing è il framework utilizzato per la creazione dell'interfaccia utente grafica (GUI) del gioco, dimostrando la separazione tra logica di gioco e presentazione.

- **Architettura GUI**: Il progetto implementa un client con interfaccia grafica in Java Swing, che comunica con lo stesso server del client testuale, a riprova del disaccoppiamento della logica di gioco dalla sua presentazione.

- **Componenti GUI**: La classe GameUI (adventure.ui/GameUI.java) è la finestra principale del gioco, un JFrame, che visualizza lo stato corrente (nome e immagine della stanza), l'output testuale e accetta l'input dell'utente. Altre finestre di dialogo specifiche, come ImageDialogUI, InventoryDialogUI, e ShowHelpCommandsDialogUI, sono implementate come JDialog per visualizzare immagini, l'inventario o i comandi di aiuto.
- **Sviluppo con NetBeans**: Lo sviluppo della GUI ha beneficiato dell'IDE NetBeans, che offre strumenti per facilitare la creazione di interfacce utente, come il drag-and-drop di componenti e la generazione automatica di codice.
- **Gestione Eventi**: La logica di gestione degli eventi utente, come la pressione di pulsanti o l'invio di comandi testuali, è integrata nelle classi dell'interfaccia utente, spesso utilizzando listener (che possono essere implementati tramite classi anonime o, come accennato in ActionBinder, tramite espressioni lambda per eventi più complessi).

Thread e programmazione concorrente

La programmazione concorrente è stata utilizzata per gestire attività in background e migliorare l'esperienza utente senza bloccare il flusso principale del gioco.

- **AtmosphereThread**: La classe Kronox1914 (implementazione/Kronox1914.java) gestisce una AtmosphereThread che opera in un thread separato. Il suo compito è stampare messaggi atmosferici o suggerimenti a intervalli regolari (ogni 2 minuti) nella console, adattandosi all'epoca corrente del gioco. Questo migliora l'immersività senza interrompere l'interazione del giocatore.
- **MusicPlayer**: La riproduzione della musica di sottofondo è gestita dalla classe MusicPlayer (sound/MusicPlayer.java), che implementa l'interfaccia Runnable per eseguire l'audio in un thread separato. Questo garantisce che la musica venga riprodotta in modo asincrono, senza bloccare il thread principale dell'applicazione.
- **Meccanismi di Concorrenza Java**: La creazione di thread avviene implementando l'interfaccia Runnable (come in MusicPlayer e AtmosphereThread) e avviando un nuovo Thread con il metodo start().

Socket e/o REST

La comunicazione di rete nel progetto è stata implementata tramite Java Sockets, una scelta specifica rispetto ad approcci basati su REST.

- **Architettura Client-Server con Socket**: Il progetto è basato su un'architettura client-server, e la comunicazione tra questi due componenti è stata implementata utilizzando direttamente le **Java Sockets**. Il Server (adventure/Server.java) gestisce le connessioni dei client e l'esecuzione del motore di gioco per ciascun client, mentre il Client (adventure/Client.java) invia l'input dell'utente al server e visualizza l'output ricevuto.
- **Motivazione della scelta (Sockets vs. REST)**: La scelta di usare Java Sockets è stata motivata dalla necessità di una **comunicazione bidirezionale persistente**, tipica di un gioco interattivo, che beneficia di un canale costante per lo scambio di dati tra client e server. Questa si contrappone al modello stateless di HTTP/REST. L'uso di Sockets ha anche fornito un **controllo a basso livello** e flessibilità sul protocollo di comunicazione, consentendo di definire un protocollo personalizzato e

snello, ottimizzato per lo scambio di comandi e messaggi specifici del gioco, riducendo l'overhead di rete rispetto a un framework RESTful.

- **Implementazione**: Il server utilizza ServerSocket per ascoltare le richieste di connessione, e il client utilizza Socket per inizializzare la connessione. Una volta stabilita la connessione, si ottengono InputStream e OutputStream per lo scambio di dati, un modello simile all'I/O su stream di oggetti.

Appendice

Di supporto a questa documentazione di "KRONOX1914" ci sono anche:

- **Guida per l'utente**: (manuale_utente.md): che contiene la spiegazione dei comandi utilizzabili durante il gioco e su come avviare il gioco stesso.
- **Java Doc**: all'interno di ciascuna classe *java* all'interno del codice sorgente.
- **Soluzione del gioco**: (soluzione.md): contenente le mappe complete del gioco e la sequenza di mosse per iniziare e terminare il gioco.